

TDD: Modernização do transporte Java ↔ Pascal (SDK oserver)

Campo	Valor
Status	Draft para revisão
Autor	Gatti (CTO)
Data	22/07/2026
Escopo	SCCI Java Híbrido, Launcher Java, programas "w" do oserver
Decisão pendente	Transporte da v2 do SDK (alternativas 2, 3 ou 4)

1. Contexto e problema

O monólito modular Java (SCCI Java Híbrido) precisa invocar programas Pascal legados durante a migração via Strangler Fig. A implementação atual (v1) usa JNA para chamar uma lib C que faz `socketpair(AF_UNIX) + posix_spawn`, entregando ao programa Pascal um FD bidirecional no número 6. O programa é invocado como `programa 6 6 <ip>`.

Problemas identificados na v1:

- Pinning de Virtual Threads.** As chamadas bloqueantes `read()`, `waitpid()` e a seção serializada por `SPAWN_LOCK` acontecem em código nativo. Cada chamada em espera prende uma carrier thread. Com pool default de carriers (1 por core), N chamadas Pascal simultâneas congelam todas as Virtual Threads da JVM, incluindo endpoints sem relação com Pascal.
- Race de vazamento de FD.** O `socketpair` cria FDs sem `SOCK_CLOEXEC` e o `fcntl` vem depois. Spawn concorrente de outra thread herda os FDs. A ponta vazada mantém o socket aberto, o EOF nunca chega e a chamada pendura até o timeout.
- Resposta parcial tratada como sucesso.** Timeout no meio da leitura retorna bytes parciais como resposta válida. Risco de corrupção silenciosa de dados em fluxo de crédito.

4. **errno não verificado.** `read` retornando -1 é sempre tratado como timeout. EINTR e ECONNRESET viram falso timeout ou truncamento.
5. **Sem timeout de escrita e sem deadline total da operação.**
6. **chdir global** durante o spawn afeta todas as threads do processo em caso de falha na restauração.

O protocolo atual usa framing por EOF nos dois sentidos: o pai escreve o request, faz `shutdown(WR)`, e a resposta termina quando o programa fecha o socket. Consequência: um processo por requisição está gravado no protocolo. Worker persistente exigiria mudar o framing nos programas Pascal, o que está fora de escopo nesta fase.

2. Objetivos

- Eliminar código nativo (JNA e lib C) do caminho de dados do SDK.
- Eliminar a classe de incidente de pinning de Virtual Threads.
- Manter os programas Pascal intocados, ou com mudança mínima concentrada em um único componente.
- Manter a interface pública do SDK estável. A troca é interna ao adapter `SdkPascalAdapter`.
- Gerar dados empíricos de latência (P50, P99) e comportamento sob carga para decidir entre alternativas.

3. Não-objetivos

- Worker pool persistente (exige mudança de protocolo nos programas Pascal). Fica como fase futura, guiada por métricas por operação.
- Substituir o caminho gRPC do Launcher Java. Ele permanece como segundo adapter da porta hexagonal.
- Migrar lógica de negócio Pascal para Java. Isso é o Strangler Fig em si, não este TDD.

4. Alternativas avaliadas

Alternativa 1: JNA + lib C (atual, v1)

Mantida apenas como legado até a substituição. Recebe dois fixes obrigatórios (seção 8). Não é candidata para a v2.

Alternativa 2: ProcessBuilder + stdin/stdout

O JDK cria o processo e conecta pipes automaticamente. O Java escreve o request em `process.getOutputStream()`, fecha o stream (equivalente ao `shutdown(WR)`) e lê `process.getInputStream()` até EOF.

- Condição: o programa aceitar `programa 0 1 <ip>`, ou seja, o `InitFD` tratar os dois argumentos como FDs distintos de leitura e escrita.
- Virtual Threads: I/O de pipes de `Process` não desmonta a VT. Mitigação: `ExecutorService` de platform threads dedicado, dimensionado pelo paralelismo alvo de chamadas Pascal.
- Esforço: mínimo. API padrão do JDK, ~80 linhas.

Alternativa 3: Unix Domain Socket em Java puro (JDK 16+)

O Java abre `ServerSocketChannel` em um path (`StandardProtocolFamily.UNIX`), sobe o programa via `ProcessBuilder` passando o path como argumento, o programa conecta e o `accept()` devolve o canal bidirecional.

- Virtual Threads: `SocketChannel` é integrado ao scheduler do Loom. Read bloqueado estaciona a VT e libera a carrier. Zero pinning, sem executor dedicado.
- Condição: alterar o lado Pascal para conectar ativamente no path. Conflita com a restrição de não tocar nos programas.
- Esforço: médio no Java, alto no parque Pascal. Candidata apenas se combinada com o shim (alternativa 4).

Alternativa 4: UDS + shim

Resolve o conflito da alternativa 3. O shim é um executável mínimo (~30 linhas, C ou Pascal) lançado no lugar do programa. Ele conecta no path UDS recebido como argumento, faz `dup2()` do socket para o FD 6 e executa o programa original via `exec`. O programa Pascal encontra o FD 6 pronto, idêntico ao socketpair atual, sem uma linha alterada.

- Virtual Threads: lado Java idêntico à alternativa 3. Zero pinning.
- Esforço: baixo. Shim escrito uma vez, serve para todos os programas.

Matriz de decisão

	Código nativo	Muda Pascal	Virtual Threads	Esforço	Risco residual

1. JNA (atual)	Sim	Não	Pina	Zero	Alto (bugs + pinning)
2. stdin/stdout	Não	Não*	Executor dedicado	Mínimo	Baixo
3. UDS puro	Não	Sim	Nativo	Médio/Alto	Baixo
4. UDS + shim	Só o shim	Não	Nativo	Baixo	Baixo

*Se o InitFD aceitar FDs distintos. A PoC 0 responde isso.

5. Arquitetura alvo

O domínio conhece apenas a porta `LegacyCalcPort` . Adapters:

- `SdkPascalAdapter` : transporte local (v1 hoje, v2 após decisão deste TDD).
- `GrpcPascalAdapter` : Launcher Java dedicado.

Resolver de transporte por operação, via configuração externalizada:

```
legado:
operacoes:
  calculo-amortizacao: sdk
  emissao-cci: grpc
  validacao-garantia: sdk
```

Requisitos transversais dos adapters:

1. Contrato idêntico: mesma semântica de timeout, mesma taxonomia de erro, mesmo comportamento de retry. O domínio não percebe qual transporte atendeu.
2. Fallback automático sdk → grpc apenas para operações idempotentes (leitura e cálculo puro). Nunca para emissão de documento ou gravação contábil.
3. Métricas com tag `transport=sdk|grpc` e `operacao=<nome>` desde o dia um. P50, P99, taxa de erro, timeouts.
4. Deadline absoluto por operação, cobrindo escrita, espera e leitura.

Topologia de deployment conforme diagrama ArchStudio de 22/07: single instance para on-premise, múltipla instância para SaaS com instância Java pura (Fargate) como bulkhead. Correção pendente no diagrama: a zona de pinning inclui o SCCI Java Híbrido enquanto a v1

(JNA) estiver ativa.

6. Plano de PoCs (execução via Claude Code / Fable)

Cada PoC abaixo está especificada como tarefa autônoma para Claude Code, com entradas, saídas e critério de aceite. Sugestão de estrutura de repositório:

```
poc-sdk-pascal/
  CLAUDE.md          # contexto e convenções para o agente
  harness/           # benchmark e teste de pinning (comum a todas)
  mock-pascal/       # programa Free Pascal que emula os programas w
  alt2-stdio/        # ProcessBuilder stdin/stdout
  alt3-uds/          # UDS Java puro
  alt4-shim/         # shim + UDS
  baseline-jna/       # v1 isolada para comparação
  baseline-grpc/      # sidecar gRPC mínimo
  resultados/        # relatórios gerados
```

Conteúdo sugerido do `CLAUDE.md` do repositório:

```
# Contexto
PoCs de transporte Java <-> Pascal para o SDK oserver da Prognum.
Contrato do programa Pascal: recebe argv[1]=FD leitura, argv[2]=FD escrita,
argv[3]=IP. Framing por EOF: request termina com shutdown/close do lado
escritor, resposta termina quando o programa fecha a saída.

# Convenções
- Java 21+, Spring não é necessário nas PoCs, apenas JDK puro e JMH.
- Free Pascal (fpc) para o mock. Compilar com fpc -O2.
- Toda PoC expõe a mesma interface: byte[] exchange(byte[] request, long timeoutM
- Resultados em resultados/<poc>/relatorio.md com P50, P99, throughput e
  observações de pinning.
- Não usar JNA fora de baseline-jna/.

# Comandos
- ./gradlew :harness:jmh    (benchmarks)
- ./gradlew :harness:pinningTest  (teste de saturação de VTs)
```

PoC 0: teste do contrato InitFD (bloqueante para as demais)

Pergunta: o InitFD dos programas w aceita FDs distintos (0 1) ou exige FD único bidirecional (6 6)?

Tarefa para o Fable:

1. Escolher 3 programas w reais de perfis diferentes: um de cálculo puro, um que acessa banco, um que gera documento.
2. Para cada um, executar no shell: `programa 0 1 127.0.0.1 < request.bin > response.bin`, usando um request capturado de produção ou gerado pelo caminho atual.
3. Comparar `response.bin` byte a byte com a resposta obtida pela v1 (JNA) para o mesmo request.
4. Repetir com `socat UNIX-LISTEN:/tmp/t.sock EXEC:'programa 6 6 127.0.0.1',fdin=6,fdout=6` para validar o caminho do shim.

Critério de aceite: relatório indicando, por programa, se 0 1 funciona, se apenas 6 6 funciona, ou se nenhum funciona fora do socketpair. Este resultado decide entre alternativa 2 e alternativa 4.

Esforço estimado: meio dia. Requer acesso a um ambiente com os binários e banco de homologação.

PoC 1: mock Pascal + harness de benchmark

Objetivo: infraestrutura comum para comparar todas as alternativas em igualdade de condições, sem depender do ambiente Prognum.

Tarefa para o Fable:

1. Escrever `mock-pascal/wmock.pas` em Free Pascal que reproduz o contrato: lê os FDs de argv, lê o request até EOF, simula processamento com sleep configurável via env (`WMOCK_DELAY_MS`), escreve uma resposta de tamanho configurável (`WMOCK_RESP_BYTES`) e fecha. Deve funcionar tanto com FDs distintos quanto com FD único, espelhando o comportamento real descoberto na PoC 0.
2. Escrever o harness com JMH: cenários de request 1KB/64KB, resposta 1KB/256KB, delay 0ms/50ms/500ms, concorrência 1/8/64/512.
3. Métricas: P50, P99, P99.9, throughput, tempo de spawn isolado.

Critério de aceite: `./gradlew :harness:jmh` roda contra qualquer módulo alt* e gera relatório comparável em `resultados/`.

PoC 2: teste de pinning (o mais importante)

Objetivo: provar em código o comportamento das Virtual Threads em cada alternativa. Este é o argumento decisivo para o time.

Tarefa para o Fable:

1. No harness, criar `pinningTest` : sobe a JVM com -
`Djdk.virtualThreadScheduler.parallelism=4 -Djdk.tracePinnedThreads=full`.
2. Disparar 64 chamadas Pascal concorrentes com delay de 2s (mock segurando a resposta).
3. Em paralelo, disparar 1000 Virtual Threads triviais (incremento de contador com sleep de 1ms) e medir a latência delas durante a janela.
4. Executar contra baseline-jna, alt2, alt3 e alt4.

Critério de aceite: relatório mostrando, por alternativa: quantas carriers ficaram presas, latência das VTs triviais durante a saturação, e output do `tracePinnedThreads`. Expectativa: baseline-jna congela as VTs triviais, alt3/alt4 não sofrem, alt2 protegida pelo executor dedicado.

PoC 3: alternativa 2 (ProcessBuilder stdin/stdout)

Tarefa para o Fable:

1. Implementar `Alt2Exchange` com `ProcessBuilder`: write no `OutputStream`, close, read até EOF, deadline absoluto via `Future` com timeout, `destroyForcibly` + `waitFor` no cleanup.
2. Executor dedicado de platform threads, tamanho configurável, com fila limitada e timeout de aquisição.
3. Tratar os casos de erro: processo morre antes de ler tudo, resposta vazia, timeout parcial (sempre erro, nunca resposta parcial).

Critério de aceite: passa no harness e no pinning test. Diff de comportamento contra baseline-jna documentado.

PoC 4: alternativa 4 (UDS + shim)

Tarefa para o Fable:

1. Escrever o shim em C (~30 linhas): `connect` no path UDS de `argv[1]`, `dup2` para o FD 6, `execv` do programa de `argv[2]` em diante. Versão alternativa em Pascal se o time preferir manter stack única.
2. Implementar `Alt4Exchange` : `ServerSocketChannel` UDS com path único por chamada (UUID), permissão 0700, spawn do shim via `ProcessBuilder`, `accept` com timeout, handshake com token passado via env para garantir que quem conectou é o filho

esperado, unlink do path após accept.

3. Chamadas de I/O direto em Virtual Threads, sem executor dedicado.

Critério de aceite: passa no harness e no pinning test com zero pinning. Funciona com o wmock em modo FD 6.

PoC 5: baseline gRPC

Objetivo: dar número real ao overhead do caminho gRPC, hoje estimado em 100 a 500µs.

Tarefa para o Fable:

1. Servidor gRPC mínimo (um serviço `Exchange(bytes) returns (bytes)`) que internamente chama a melhor alternativa local vencedora das PoCs 3/4.
2. Cliente no harness medindo o round trip completo via loopback.

Critério de aceite: relatório comparando P50/P99 do caminho local vs gRPC no mesmo hardware. Alimenta a decisão de flag por operação.

PoC 6: validação com programas reais da Prognum

Objetivo: repetir o harness contra 3 programas w reais em homologação, com a alternativa vencedora.

Tarefa para o Fable:

1. Adaptar o harness para requests reais capturados (anonimizados se contiverem dado de cliente).
2. Rodar carga sustentada de 30 minutos, medir P99, taxa de erro, vazamento de FDs (`ls /proc/<pid>/fd | wc -l` ao longo do tempo) e zumbis (`ps -o stat`).
3. Comparar respostas byte a byte com a v1 para os mesmos requests (teste de equivalência).

Critério de aceite: equivalência de respostas 100%, zero vazamento de FD, zero zumbi, P99 igual ou melhor que a v1.

Sequência e dependências

```
PoC 0 (contrato) —> decide alt2 vs alt4
PoC 1 (harness)  —> pré-requisito de 2 a 6
PoC 2 (pinning)  —> argumento decisivo, roda cedo
PoC 3 e 4        —> em paralelo após PoC 1
PoC 5 (gRPC)     —> após vencedora local definida
```


PoC 6 (real) —> gate final antes de produção

Estimativa total: 3 a 5 dias de execução com Claude Code, mais o acesso a homologação para PoCs 0 e 6.

7. Critérios de decisão

A v2 do SDK será a alternativa que:

1. Zerar pinning no teste da PoC 2 (ou confiná-lo em executor dedicado, no caso da alt2).
2. Tiver P99 igual ou melhor que a v1 no harness.
3. Passar o teste de equivalência da PoC 6 com programas reais.
4. Não exigir mudança nos programas Pascal (alt2 ou alt4).

Empate técnico entre alt2 e alt4 resolve-se pela simplicidade: alt2 vence por usar apenas API padrão, desde que a PoC 0 confirme o contrato 0 1.

8. Fixes obrigatórios na v1 (independentes da decisão)

Aplicar imediatamente, pois a v1 segue em produção até a v2 entrar:

1. **CLOEXEC atômico:** `socketpair(AF_UNIX, SOCK_STREAM | SOCK_CLOEXEC, 0, sv)` (valor `0x80001`). Elimina a race de herança de FD entre threads. O `adddup2` do spawn limpa a flag para o filho correto.
2. **Resposta parcial é erro:** timeout com bytes já recebidos lança `IOException`. Nunca retornar payload truncado.
3. Recomendados na sequência: verificação de `errno` via `Native.getLastError()` distinguindo `EAGAIN` de `EINTR/ECONNRESET`, timeout de escrita, deadline absoluto da operação.

9. Riscos e mitigações

Risco	Impacto	Mitigação
PoC 0 revelar que nenhum programa aceita FDs alternativos nem FD 6 via shim	Bloqueia alt2 e alt4	Improvável (o contrato <code>argv</code> sugere flexibilidade). Plano C: manter v1 corrigida com executor dedicado e priorizar caminho gRPC

Programas com estado dependente de cwd	Respostas erradas na alt2/alt4	ProcessBuilder.directory() por processo, testado na PoC 6
Diferença sutil de comportamento de buffer entre pipe e socket	Timeout ou truncamento em respostas grandes	Cenário de resposta 256KB no harness (PoC 1)
Spawn de 1 a 5ms dominar o P99 em operações de alta frequência	Meta de latência não atingida	Fora de escopo desta fase. Métricas por operação identificarão candidatas a worker persistente (fase futura)

10. Fases futuras (fora deste TDD)

- Worker persistente com mudança de framing (length-prefixed), guiado pelas métricas por operação. Candidato: worker host único linkando units compartilhadas.
- Migração alt2 → alt4 se o volume de chamadas concorrentes tornar o executor dedicado um gargalo.
- Aposentadoria do transporte SDK conforme módulos Pascal migram para Java (métrica de tráfego da instância híbrida como medidor do Strangler Fig).